Kasi Yedumati · Full Stack Training

Lesson 8 — Authentication

Course: FastAPI for Full Stack · **Session 8 of 10** · Learning path · **Previous:** Lesson 7 · **Next:** Lesson 9

> **Module Outcome:** You can log in with an email and password, call `/me` with the token, and block a delete when the caller is not HR.

You will build	`POST /login`, `GET /me`, and a delete route that checks the role
New words	Authentication, authorization, hash, token
You need	The project split into `database.py`, `models.py`, `schemas.py`, and `routers/`

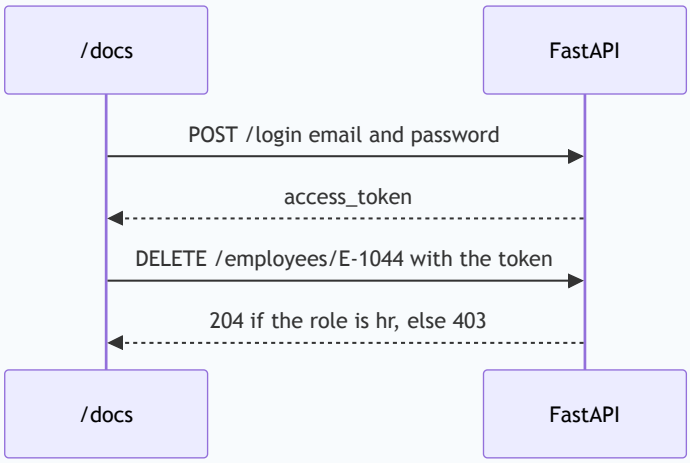
From the requirements. HR manages employee records. An employee may see the directory. Only HR may change or delete a record. The password is stored as a hash, never as the text the person typed.

Practice logins, seeded when the user table is empty:

Email	Password	Role
`ananya.rao@acme.example`	`Acme#1042`	employee
`rahul.sharma@acme.example`	`Acme#1045`	hr

These passwords are for this practice API only.

1. Login returns a token



Authentication answers "who is this?". **Authorization** answers "is this person allowed?". Login is authentication. The HR check on delete is authorization.

```
pip install bcrypt PyJWT
```

`bcrypt` turns a password into a hash. `PyJWT` builds the token. The new files in `examples/lesson08/` are `security.py`, `routers/auth.py`, and the `User` class in `models.py`.

```
class User(Base):
    __tablename__ = "app_user"

    id = Column(Integer, primary_key=True)
    email = Column(String(120), unique=True, nullable=False)
    password_hash = Column(String(255), nullable=False)
    role = Column(String(20), nullable=False)
```

`password_hash` is the column. There is no column named `password`.

```
def hash_password(plain: str) -> str:
    return bcrypt.hashpw(plain.encode(), bcrypt.gensalt()).decode()

def create_token(email: str, role: str) -> str:
    payload = {
        "sub": email,
        "role": role,
        "exp": datetime.now(timezone.utc) + timedelta(minutes=60),
    }
    return jwt.encode(payload, SECRET_KEY, algorithm="HS256")
```

`SECRET_KEY` comes from `.env`. `acmehr-class-secret-change-me-please` is only a practice value. `exp` makes the token expire after 60 minutes.

`POST /login` reads the form fields `/docs` sends. The box labelled **username** is the email.

```
@router.post("/login")
def login(form: OAuth2PasswordRequestForm = Depends(), db: Session = Depends(get_db)):
    user = db.query(User).filter(User.email == form.username).one_or_none()
    if user is None or not verify_password(form.password, user.password_hash):
        raise HTTPException(status_code=401, detail="Wrong email or password")
    return {"access_token": create_token(user.email, user.role), "token_type": "bearer"}
```

In `/docs`, authorize with Rahul's email and password. Then call `GET /me`.

Output:

```
{"email":"rahul.sharma@acme.example","role":"hr"}
```

Call `GET /me` with no token. **401**.

---

2. Delete checks the role

```
def require_hr(user: User = Depends(get_current_user)):
    if user.role != "hr":
        raise HTTPException(status_code=403, detail="HR only")
    return user
```

`get_current_user` reads the token, checks the signature, and loads the user. A bad or missing token is **401**. A real user with the wrong role is **403**.

`DELETE`, `PUT`, and `PATCH` on `/employees/{employee_id}` each take `Depends(require_hr)`. Anything that changes a record is HR's job. A PATCH with no token is **401**, so nobody can rename Ananya anonymously.

```
@router.patch("/{employee_id}", response_model=EmployeeOut)
def update_employee(
    employee_id: str,
    changes: EmployeeUpdate,
    db: Session = Depends(get_db),
    user=Depends(require_hr),
):
```

Log in as Ananya and delete `E-1044`. **403**. The row stays. Log in as Rahul and delete a person you created for the exercise. **204**.

Common error: You type the email into a field named email. On `/docs` the login form calls it **username**. Put the email there.

Common error: The password in MySQL is the text `Acme#1045`. Then a leak of the table reveals the password. Store `password_hash` from `hash_password`.

Try these

1. **(Easy)** `GET /me` with no lock icon on `/docs`. Status **401**.
2. **(Easy)** Log in as Ananya. `/me` shows `"role": "employee"`.
3. **(Medium)** As Ananya, DELETE an employee. **403**. As Rahul, the same delete on a test row is **204**.
4. **(Stretch)** Change `SECRET_KEY`, restart, and call a route with the old token. Status **401**.

Interview: *What is the difference between 401 and 403?* — 401 means we do not accept the token. 403 means we know who you are, and this role may not do that action.

Key takeaways

- The table stores a hash, not the password.
- Login returns a token. Later requests send `Authorization: Bearer <token>`.
- `/me` proves the token. `require_hr` proves the role.
- A wrong password is **401**. A non-HR delete, replace, or update is **403**.

Lab — Ticket ACME-208

Homework

Next: Frontend and CORS. A registration page in the browser calls this API.